



Segurança em Redes de Computadores (Computer Network Security)

(MIETI 4º Ano/S2 - 6707N5)

Applied cryptography

Henrique Santos (hsantos@dsi.uminho.pt)

Dpt. Sistemas de Informação

Ext. 510302

Summary

- Applied cryptography
 - Concepts and terminology
 - Basic Protocols
 - Public key protocol
 - Email specific protocols (PGP and S/MIME)
 - Digital Signatures
 - Digital certificates
 - PKI and Certificate Management
 - Trust chains
 - Kerberos as an example of an authentication protocol

Concepts and terminology

■ Terminology and fundamentals

- **Plain Text** or **Message** (M)
- **Ciphered Text** (C) a message after being processed by an **Encryption algorithm** (E)
- **Decryption** (D) – the process to recover the original message from a ciphered text
- $E(M) = C$ while $D(C) = M$
- $D(E(M)) = M$ should be always true
- **Cryptography** – Art and Science to keep information secrecy
- **Cryptanalysis** – Art and Science to break Cryptography
- **Cryptology** – Mathematical fundamentals for Cryptography and Cryptanalysis.

Conceitos e Terminologia

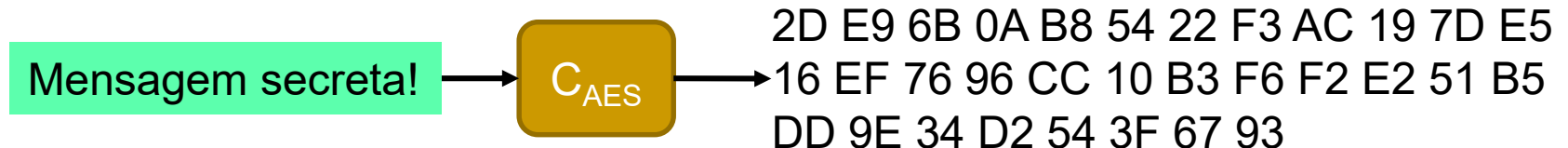
- Terminology and fundamentals (cont)
 - What cryptography promotes?
 - **Confidentiality; Integrity; Authenticity and non-repudiation**
 - Attacks (against cryptography)
 - Based only on ciphered text :-(
 - Require to deduce a key and to recognize a message
 - Based on message knowledge :-|
 - Based on message choice :-)
 - Variant of the last case, when the attacker is able to submit messages to the cryptography algorithm!

Concepts and terminology

■ Terminology and fundamentals (cont.)

□ Cryptographic Functions

- **Restrict Algorithms** – security level depends only of the algorithm
- **Hash Functions**
- **Key Algorithms** – security level depends on a key used by the algorithm. Keyspace is the range of possible key values

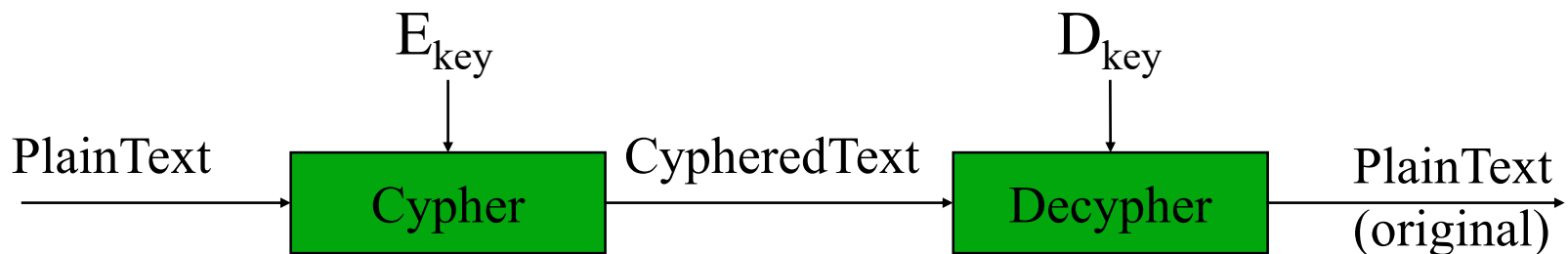


Basic Protocols

■ Key based protocols

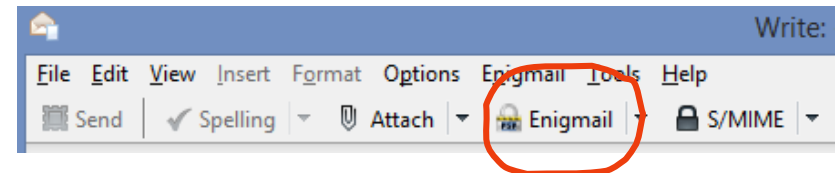
□ Key Algorithms

- Assuming there exists $E_k(M) = C$ and $D_k(C) = M$
- **Symmetric Algorithm** (**secret key**) – key previously negotiated; **very fast**; these algorithms are commonly divided into two classes: Block Ciphers (most common) and Stream Ciphers
- **Public Key Algorithms** (asymmetric algorithms) – use two unrelated keys; one (**public**) to cipher and the other (**private**) to decipher; **very computational intensive and slow**

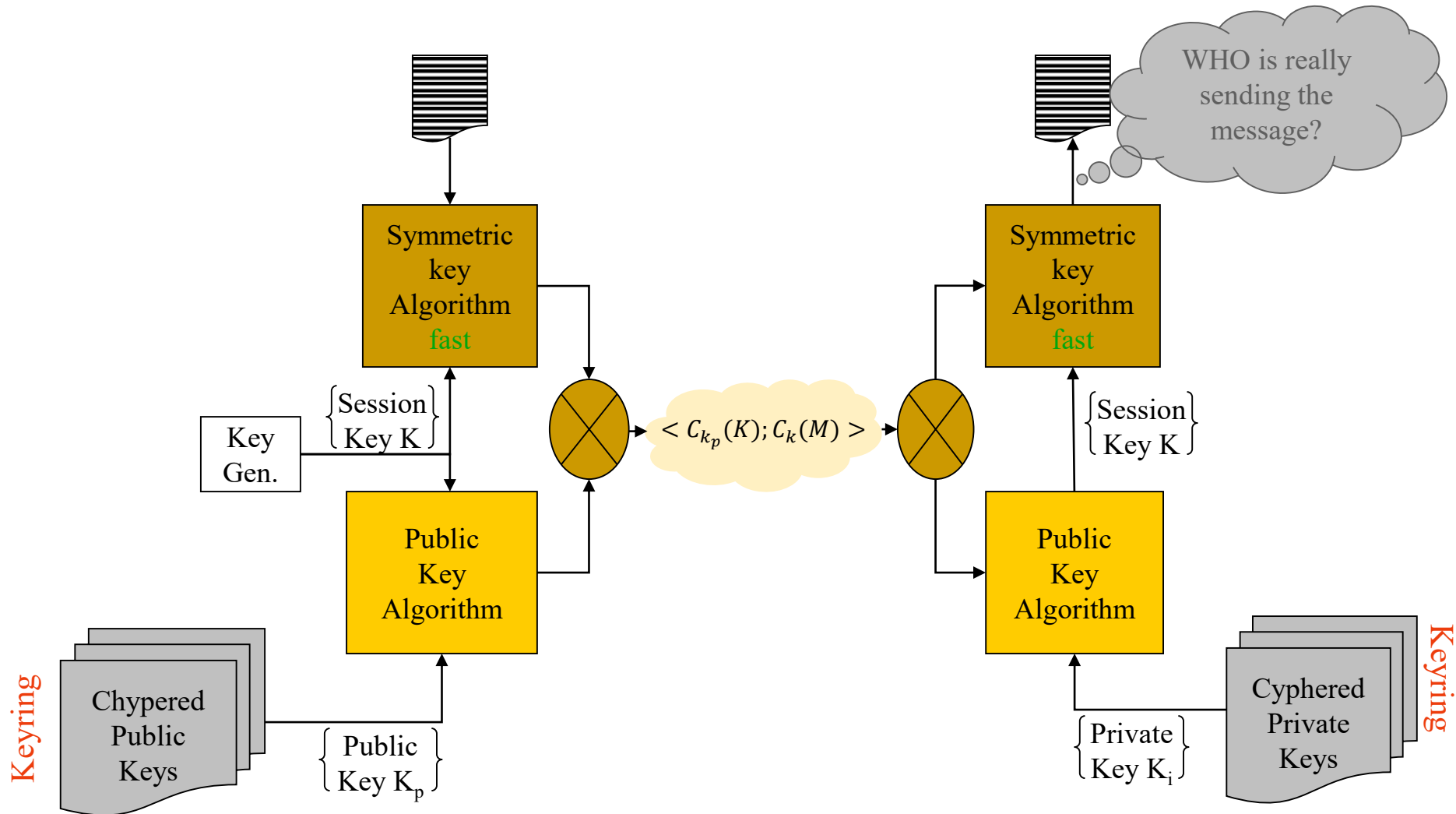


PGP – *Pretty Good Privacy*

- An application to promote privacy and security of e-mail communications (due to Phil Zimmermann, 1991).
 - ❑ Based on OpenPGP standard defined by RFC 4880
 - ❑ Integrates with several Operating Systems
 - ❑ Integrates with several e-mail applications
 - ❑ Includes
 - *Plug-ins* for several e-mail clients
 - Stand-alone application for:
 - ❑ Key pair generation
 - ❑ File encryption and decryption
 - ❑ Secure file deletion
 - ❑ Virtual Private Network (VPN) support
 - ❑ ...



PGP – operation principles

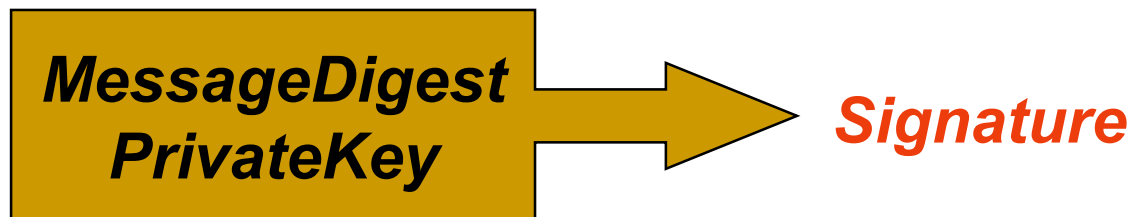


Who is sending the message?

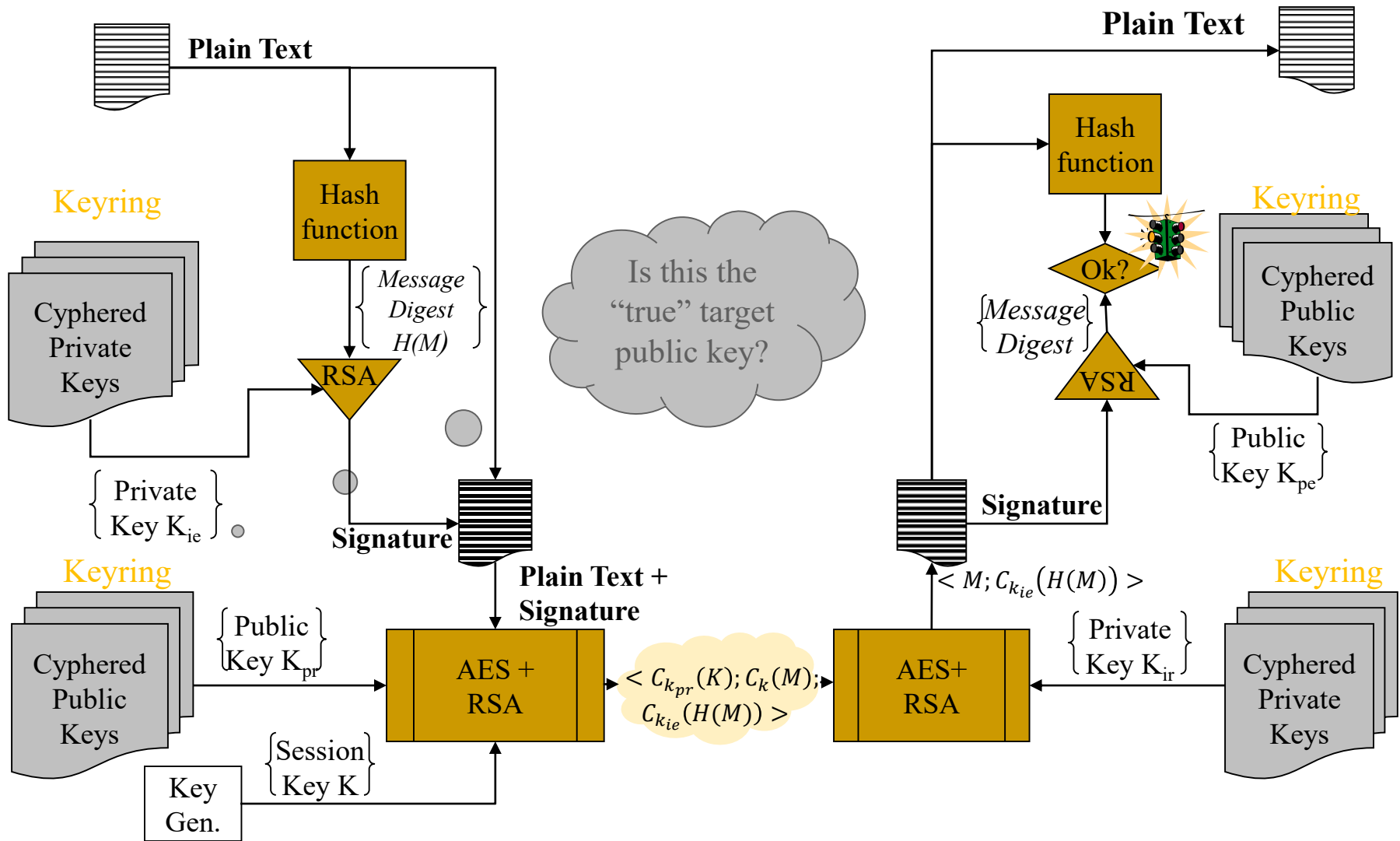
- We can use a third party – arbitrator – in which we completely trust...
- ... but it would be better to provide some sort of *Digital Signature*, assuring:
 - The sender authenticity (**Authentication**)
 - Message was not modified (**Integrity**)
 - The sender will not deny the fact (**Non-repudiation**)
- Encryption with private key serves the purpose – on target, deciphering with the associated public key, proves the "identity" of the issuer.

Digital Signatures

- Encrypt the entire document with the private key requires a lot of processing time...
- A Hash function applied to a message produces a binary pattern (for example, 160 bits) “uniquely” representing the message



Digital Signatures



Digital Certificates

- Who Is (“absolutely”) the owner of a given public key?
 - Use only keys that are distributed manually (limited to local applications) – manual public key distribution manual
 - Look for a **Digital Certificate** (or **Cert**)
- A digital certificate typically includes:
 - **Name of the certified entity (ID)**
 - **Public key**
 - Identification of the certification “authority”
 - Serial number
 - Date of issue and date limit
 - A self-signature
 - Other certification information (ID, photo, etc.)

S/MIME

- S/MIME (Secure/Multipurpose Internet Mail Extensions)
 - ❑ On an IETF Standards Track, using IETF MIME specification (like PGP!)
 - ❑ **Based on X.509 certificates**
 - ❑ Initially based on the PKCS#7 de facto industry standard secure message format; nowadays using the IETF Cryptographic Message Syntax (CMS), which allows advanced electronic signatures
 - ❑ End-to-end encryption (like PGP)

Digital Certificate: types

- Digital Certificate classes (different applications impose different requirements)
 - Certificates for e-mail applications
 - Certificates for Browsers
 - Certificates for SSL based services
 - Certificates for Digital Signatures
 - ...

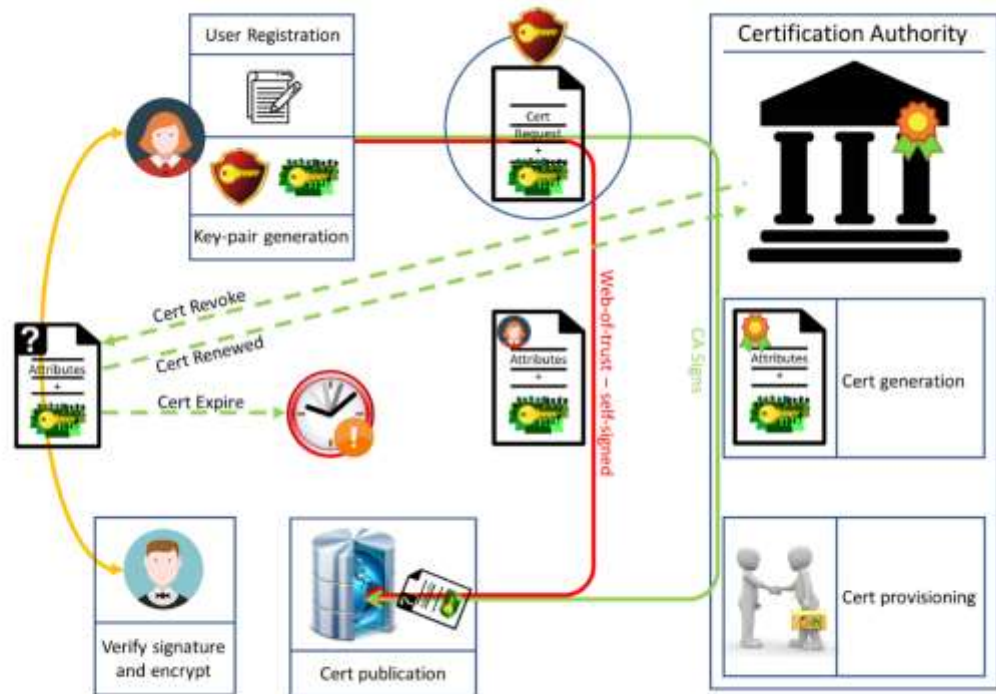
Digital Certificates: distribution

- **Certificate Servers** – simple storage repositories
- Dedicated infrastructures, designated by **PKI** (**Public Key Infrastructure**) – which offer an extended service of certificate management
 - **Certification Authority** (CA):
 - Assigning
 - Revocation
 - Storing
 - Recovering
 - "Borrow" trust!!!

Digital Certificates - PKI

“a set of *roles*, *policies*, and *procedures* needed to *create*, *manage*, *distribute*, *use*, *store*, and *revoke* digital certificates and manage public-key encryption”

Who really trusts a CA ?



Certification “Authorities”

- [CAcert](#) - A community driven certificate authority that issues certificates to the public at large for free.
- [Catsdeep FreeSSL](#) - Provider of free SSL certificates to be used for testing/debugging only.
- [Comodo](#) - Provider of 128-bit certificates and various other security related products.
- [Digicert](#) - Certificate authority offering a range of services including PKI and a seal program.
- [Digi-Sign](#) - Bulk vendor of 128-bit certificates offering PKI and verification services.
- [Digital Signature Trust Co.](#) - DST was the first CA to be licensed in the U.S. DST provides PKI-based digital certificate services and electronic commerce solutions for commercial and government clients. DST creates the trust that enables organizations to fully benefit from electronic commerce.
- [Ebizid](#) - Certificate authority offering 128 and 256 bit certificates. Also a provider of various security products including validation and vulnerability testing.
- [Enterprise SSL](#) - Provider of wildcard and single domain 128-bit Comodo SSL warranted certificates for web servers.
- [Entrust.net](#) - Subsidiary of Entrust, Inc. Sells digital certificates that enable Web servers to establish Secure Sockets Layer (SSL) sessions.
- [GeoTrust](#) - Vendor of SSL certificates that features an identity verification program.
- **[GlobalSign](#) - The Leading European Trusted Network of Certification Authorities.**
- [LiteSSL](#) - Provides secure 128/256 bit ssl digital certificates.
- [Network Solutions: SSL Certificates](#) - Offer digital certificate services and secure seal schemes for websites.
- [Pink Roccade PKI](#) - Netherlands based reseller of Verisign certificates.
- [ProntoSSL](#) - Reseller of 128Bit QuickSSL Premium, RapidSSL and Geotrust Verified Domain Certificates.
- [QualitySSL](#) - A Denmark based provider of 128 bit certificates and identity assurance products.
- [Rapid SSL](#) - Provider of 256 bit single root install SSL Certificates.
- [Secure SSL](#) - Provider of Comodo 128 bit SSL certificates and web identity services.
- [SimpleAuthority certification authority](#) - A free certification authority that can be used to generate keys and certificates for secure email, client and server SSL authentication, VPN access, etc.
- [SSL Certificate Management Site](#) - Site allows users to create, download and store free SSL self-signed certificates.
- [SSL.com](#) - Provider of wildcard, single and multi-year warranted 128-bit SSL certificates for web servers.
- [Thawte Digital Certificates](#) - Global certification authority offering a range of SSL and code signing digital certificate products.
- [The USERTRUST Network](#) - A Public Key Infrastructure providing SSL Certificates and Digital Signatures.
- **[VeriSign](#) - SSL certificates, communications and content delivery, plus security consulting.**
- [XRamp Security](#) - SSL certificates authority featuring 256 bit AES encryption.



Source: Open Directory Project

Digital Certificate: creation

- Generate a key pair
- Store **private key**
- Send **public key** to a CA
- Prove identity to the CA – **verification**
- The CA formats and signs the **certificate**
- A user requests the certificate (via email or through a web service)
- The certificate is used to demonstrate the **legitimacy** of the public key

Digital Certificates: formats

■ PGP

- ❑ PGP version;
- ❑ Public key of the certified entity (with preferred encryption algorithm: RSA, DH or DSA);
- ❑ Information concerning the certified entity (name, ID, photo,...);
- ❑ Self-signature (using the private key);
- ❑ Period of validity;
- ❑ Symmetric algorithm preference (CAST, IDEA or Triple-DES).
- ❑ (eventually) Other signatures.

Digital Certificates: formats

■ X509

- ❑ ITU-T standard (and CCITT recommendation) for a public key infrastructure (PKI) and Privilege Management Infrastructure (PMI). It specifies standard formats for:
 - Public key certificates
 - Certificate revocation lists
 - Attribute certificates
 - Certification path validation algorithm
 - ...
- ❑ Based on X.500 directory (as repository), defining a strict hierarchical system of certificate authorities (CAs) for issuing the certificates (contrasting web of trust models)

Digital Certificates: formats

• X509:

Version
Serial number
Algorithm Parameters
Issuer
Not before Not after
Subject
Subject Public Key Info
Extensions
Signature

Algorithm
Identifier

Period
of validity

Public
key

Notation used:

$CA\langle A \rangle = CA \{V, SN, AI, CA, T_v, Pk, Ap\}$

where

$Y\langle X \rangle$ = cert of X provided by Certification Authority Y

$Y\{I\}$ = Object I signed by Y. Consists of I plus a hash of I encrypted with the private key of Y.

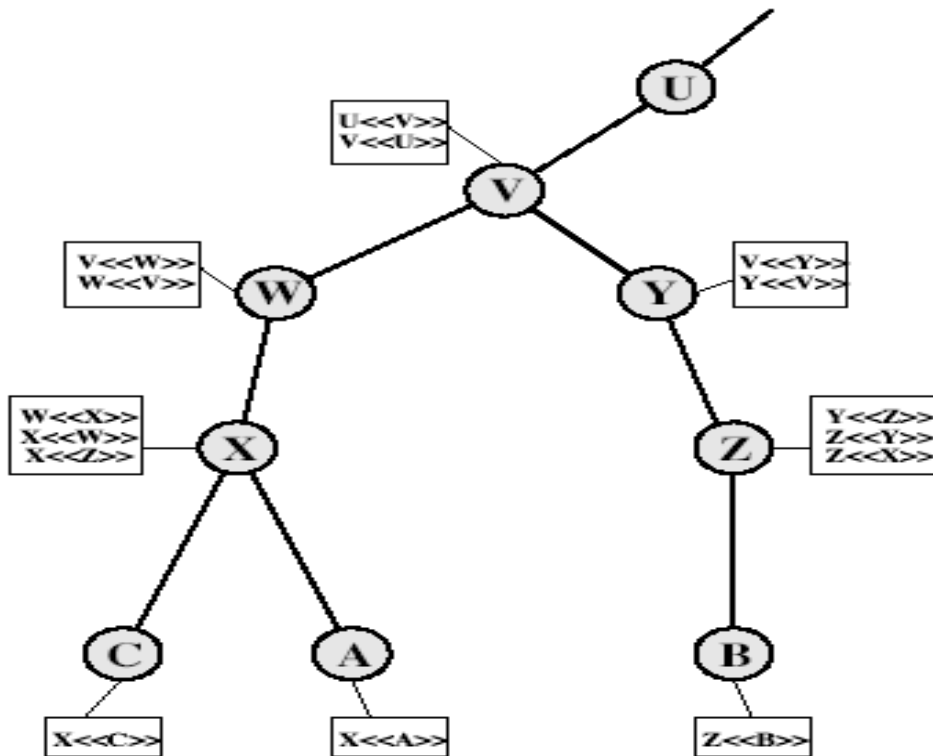
- Each extension has an unique Object Identifier (OID)
- RFC 5280: specific use of a certificate



X509: Cross-certification

- Scenario:
 - A obtained a certified provided by CA X1
 - B obtained a certified provided by CA X2
 - A can access to B's certificate, **but can not verify it**
- Solution: **X1«X2»** and **X2«X1»** (peer-to-peer cross-certification)
 - A gets B's certificate, **signed by X2**
 - A gets the X2 certificate **signed by X1**
 - Gets X2's public key and **verify B's certificate**
- Other solutions
 - Pure hierarchical
 - Bridge CA (or intermediate CA devoted to inter-CA relationship management)

X.509 Hierarchy of CAs



A gets B's certificate using the following chain:

$X \ll W \gg W \ll V \gg V \ll Y \gg$
 $Y \ll Z \gg Z \ll B \gg$

B gets A's certificate using the following chain:

$Z \ll Y \gg Y \ll V \gg V \ll W \gg$
 $W \ll X \gg X \ll A \gg$

X509: File extensions

- **.pem**: (Privacy-enhanced Electronic Mail, originally proposed for securing email using public-key cryptography, based on a unique central CA!); base64 encoded certificate, enclosed between "-----BEGIN CERTIFICATE-----" and "-----END CERTIFICATE-----"
- **.cer**, **.crt**, **.der**: usually binary versions of .pem
- **.p7b**, **.p7c**: PKCS#7 SignedData structure - no data, just certificate(s) or CRL(s); "enveloping"
- **.p12**: PKCS#12 is an archive file format for storing many cryptography objects in a single file; may contain also private keys (password protected)
- **.pfx** – PFX, predecessor of PKCS#12 (usually contains data in PKCS#12 format; e.g., PFX files generated in IIS)

Digital Certificate: trust chains

- Certification is usually provided by **trust chains**
- Several issues arise regarding CAs:
 - How "much trust" is implicit in a certificate?
 - What is the relationship between the CA and the certified entity?
 - Who can be trusted at the top-level?
 - How many levels shall be admissible in a chain?
- PGP tries to answer these questions with the concept of **introducers** – community of users who certify each other

Trust Models

- Monopoly
- Monopoly plus registration authorities
- Delegated Certification Authorities
- Oligarchy
- Anarchy (or Web of Trust)

X.509: Chain of Trust

End-entity Certificate

Owner's name
Owner's public key
Issuer's (CA's) name
Issuer's signature

Intermediate Certificate

Owner's (CA's) name
Owner's public key
Issuer's (root CA's) name
Issuer's signature

Root CA's name
Root CA's public key
Root CA's signature

Root Certificate

reference

sign

reference

sign

self-sign

PGP: Trust Model

- Three validity levels:
 - **Valid**
 - Marginally valid
 - **Invalid**
- Three trust levels:
 - **Complete trust**
 - Marginal trust
 - **No trustable**
- To consider a certificate valid it lacks a signature of “complete trust” or two “marginal trust”

PGP: Trust Model



■ Validity

- ❑ Manual verification!
- ❑ Using a fingerprint of the certificate (PGP generates a 160-bit hash that is associated with a certificate)
- ❑ Revocation is critical (how to disseminate rapidly?)
- ❑ **Alternative: Trust in a CA**

■ Trust Models

- ❑ Direct trust (one to one)
- ❑ “*Web*” of trust, or distributed trust
- ❑ **Alternative: Hierarchical trust**

Key ID: CA6436DF
Fingerprint: C852 C153 D972 B195 8CD4 1D08 2D75 B09D CA64 36DF

PKI risks!

- Who can use my public key?
- How secure is the computational infrastructure that supports the CA?
- What if there is more than one “António Manuel”?
- The CA is really an authority?
- Is the user integrated into a security system?
- How has the CA verified the certificate owner?
- To which extent is the handling of certificates secure?
- After all, why are we using certification? ☺

[Schneier, 2000]

KERBEROS



Kerberos consortium

■ Context:

- Open distributed systems (client/server model), where the user, at any workstation, wants to access various services.
We want to control the access and authenticate all requests. We assume that a workstation can not authenticate users in a secure way, and the medium is vulnerable

■ Main threats:

- Loss of access credentials
- Impersonalization at the workstation level (IP *spoofing*)
- Free access to network traffic (*sniffers*)

KERBEROS



- Initial decision: centralized authentication solution **based on symmetric encryption**
 - ❑ Secure
 - ❑ Trustable (critical role in the infrastructure operation)
 - ❑ Transparent to the user
 - ❑ Scalable

KERBEROS



- Initial scenario – prevent impersonation:
 - Each user has a password and each server has a (secret) key
 - There is a central **Authentication Service (AS)** where all passwords and keys are stored
- The AS authenticates each user for a specific service and returns a ticket to access the service

$E_{key_serv}\{username:service_name\}$

Client → *username:<ticket>* → *Server*

KERBEROS



- Problem: a ticket for a service can be used by someone who captures it and impersonate the legitimate user
- Solution: include in the ticket the physical address of the machine that requested it.

$E_{key_serv}\{username:ws_address:service_name\}$

Not a definite solution!
Back to this later

KERBEROS



- Problems: the ticket request requires the communication of the user password ... possibly in plain text! The generation of each ticket forces to enter the password

- Requirements:

- The password must be protected.

At login the user provides the username to the AS, which returns a special ticket encrypted with a key obtained from the user's password, **which enables their validation at the client...**

- A user should authenticate only once to be able to use the services to which he/she has access

Create a ticket server (TS), which uses a special ticket generated after authentication

$E_{key_{ts}}\{username:ws_address:ts_id\}$

Stored in the
client machine

KERBEROS



- Problem: services' tickets must be reusable, so not requiring user authentication repeatedly
- Solution: include temporal registration in tickets

$E_{key_{ts}}\{username:ws_address:ts_id:$
 $timestamp:timelimit\}$

$E_{key_{serv}}\{username:ws_address:service_name:$
 $timestamp:timelimit\}$

KERBEROS



- Problem: it is still possible to abuse a ticket, just reconfiguring a machine and using the victim username!
- Solution:
 - upon authentication, the AS provides a “**session key**” to the client and includes a copy of that key in the ticket
$$E_{key_serv}\{session_key:username:ws_address:service_name:timelimit:timestamp\}$$
 - before accessing the server, the client builds an AUTHENTICATOR
$$E_{session_key}\{username:address\}$$
that is sent with the ticket

KERBEROS



- Problem: they both can be captured!
- Solution: the authenticator can have a very limited lifetime.
- Problem: session keys are first transmitted in clear text through the network!
- Solution: encrypt the session key associated to TS's ticket, with the user's password; and encrypt the session keys for each service with the TS's session key.

KERBEROS

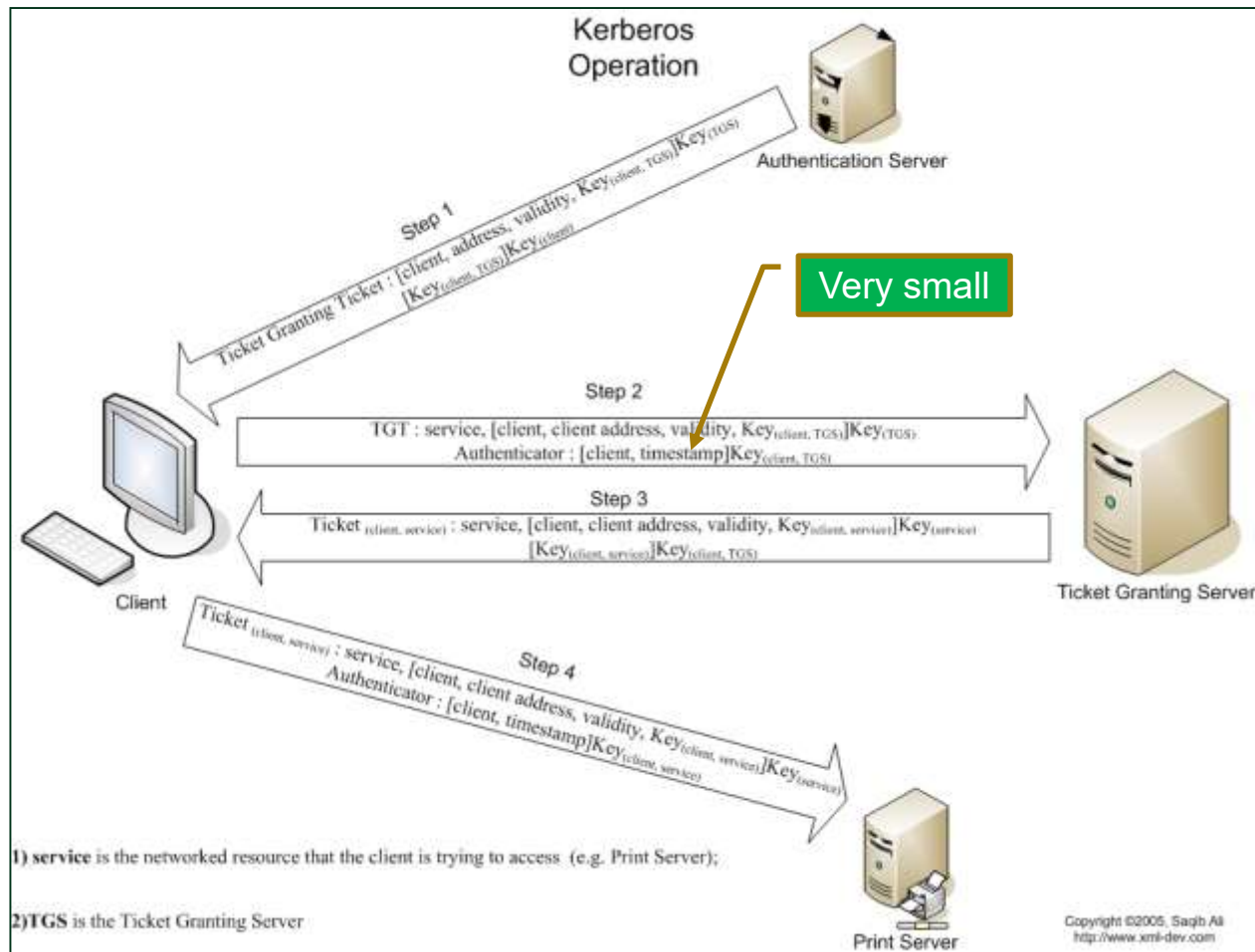


- Problem: What happens if a fake server receives the encrypted packet and ignores it ... After that it can receive all the information!
- Solution: mutual authentication => **ack** of ticket reception, encrypted with the session key (taken from the ticket), in turn encrypted with the service password.
- And so was born **Kerberos (v4)**... they say.

KERBEROS



- protocol summary



KERBEROS - summary



- Passwords are not communicated
 - All communications are encrypted
 - Limited validation period
 - Timestamps to prevent attacks by reuse
 - Mutual authentication
-
- Evolution: version 4 (using DES) has established itself as a standard for the Internet. Version 5 (not restricted to DES) addressed some security issues

KERBEROS - summary



- But...
- Single point of failure: Ticket Granting Server
- Authentication between the TGS and all servers is done through a secret key! ... **Scalability is limited**
- Timing issues can become critical (synchronous clocks and tickets' validity time)
- A workstation can save the password and use it later!
- Messages exchanged with the AS can be captured and subjected to a password attack (brute force or dictionary)

Bibliografia

- Stallings, W. Cryptography and Network Security: Principles and Practice, 5th. Prentice Hall Press, 2010. (cap. 14, 15 e 17)
- Pfleeger, C. P. and Pfleeger, S. L., Security in Computing - 4th ed., Prentice Hall, Upper Saddle River, NJ, 2007. (cap. 2)
- Menezes, A. J., P. C. v. Oorschot, et al., Handbook of Applied Cryptography, CRC Press, 2001
- Schneier, B., Applied Cryptography: Protocols, Algorithms, and Source Code in C, 2nd ed., John Wiley & Sons, Inc., 1996 (cap. 1)
- <http://www.cryptool.org/en/>
- Kaufman, C., R. Perlman, and M. Speciner, Network Security: Private Communication in a Public World, 2nd edition, Upper Saddle River, NJ 07458: Prentice Hall PTR, 2002.
- Bosworth, S. and M. E. Kabay., Computer Security Handbook, 4th ed., John Wiley & Sons, Inc., 2002
- RFC 1510, September 1993. (<ftp://ftp.isi.edu/in-notes/rfc1510.txt>)